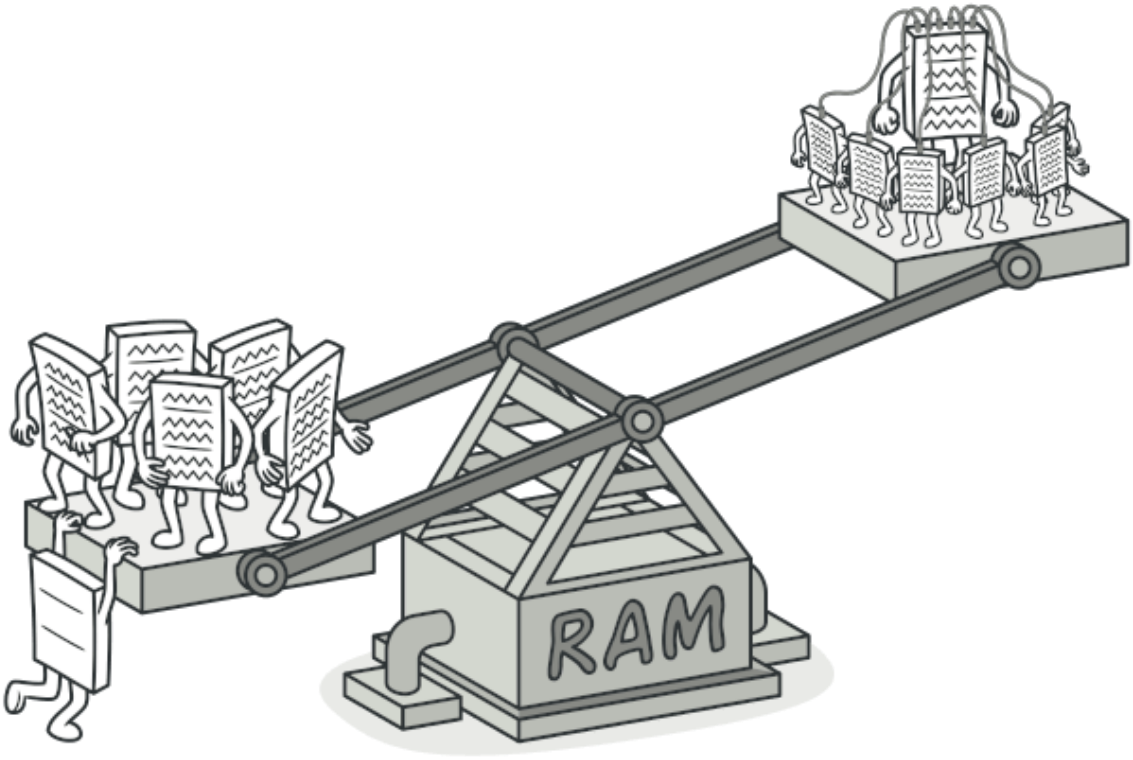
Flyweight

Also known as: Cache

Absicht

Flyweight ist ein strukturelles Entwurfsmuster, mit dem Sie mehr Objekte in den verfügbaren RAM-Speicher packen können, indem Sie gemeinsame Statusteile zwischen mehreren Objekten teilen, anstatt alle Daten in jedem einzelnen Objekt zu behalten.



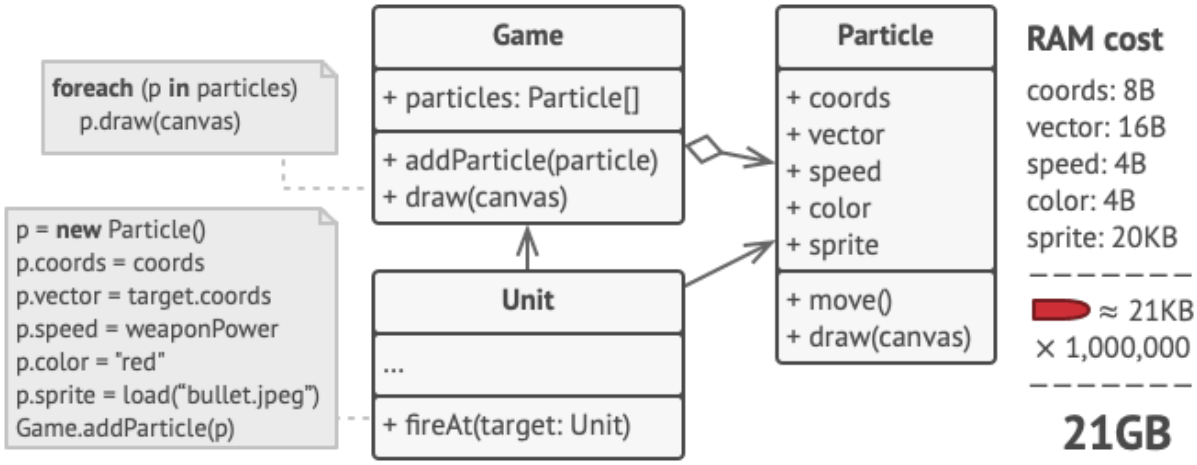
Problem

Um nach langen Arbeitsstunden etwas Spaß zu haben, haben Sie beschlossen, ein einfaches Videospiel zu entwickeln: Spieler bewegen sich auf einer Karte und schießen aufeinander. Sie haben sich für die Implementierung eines realistischen Partikelsystems entschieden und es zu einem besonderen Merkmal des Spiels gemacht. Unmengen an Kugeln, Raketen und Splittern von Explosionen sollten über die Karte fliegen und dem Spieler ein spannendes Erlebnis bieten.

Nach Abschluss hast du den letzten Commit gepusht, das Spiel erstellt und es deinem Freund zum Testen geschickt. Obwohl das Spiel auf deinem Rechner einwandfrei lief, konnte dein Freund nicht lange spielen. Auf seinem Computer stürzte das Spiel nach wenigen Minuten immer wieder ab. Nachdem du mehrere Stunden damit verbracht hattest, die Debug-Protokolle zu durchforsten, stelltest du fest, dass das Spiel aufgrund von zu wenig RAM abstürzte. Es stellte sich heraus, dass der Rechner deines Freundes

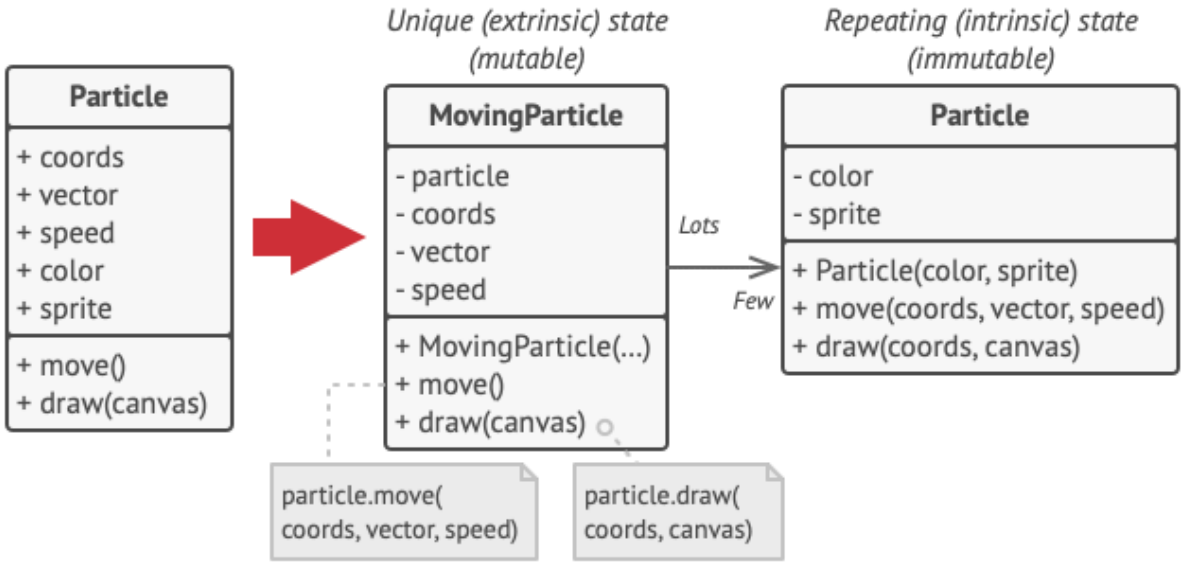
deutlich weniger leistungsstark war als dein eigener, weshalb das Problem bei ihm so schnell auftrat.

Das eigentliche Problem lag im Partikelsystem. Jedes Partikel, beispielsweise eine Kugel, ein Geschoss oder ein Granatssplitter, wurde durch ein separates Objekt mit zahlreichen Daten dargestellt. Irgendwann, als das Blutbad auf dem Bildschirm des Spielers seinen Höhepunkt erreichte, passten die neu erstellten Partikel nicht mehr in den verbleibenden RAM, sodass das Programm abstürzte.



Lösung

Bei näherer Betrachtung der `Particle`Klasse fällt Ihnen möglicherweise auf, dass die Farbe undSpriteFelder verbrauchen deutlich mehr Speicher als andere Felder. Erschwerend kommt hinzu, dass diese beiden Felder nahezu identische Daten über alle Partikel hinweg speichern. Beispielsweise haben alle Kugeln die gleiche Farbe und das gleiche Sprite.

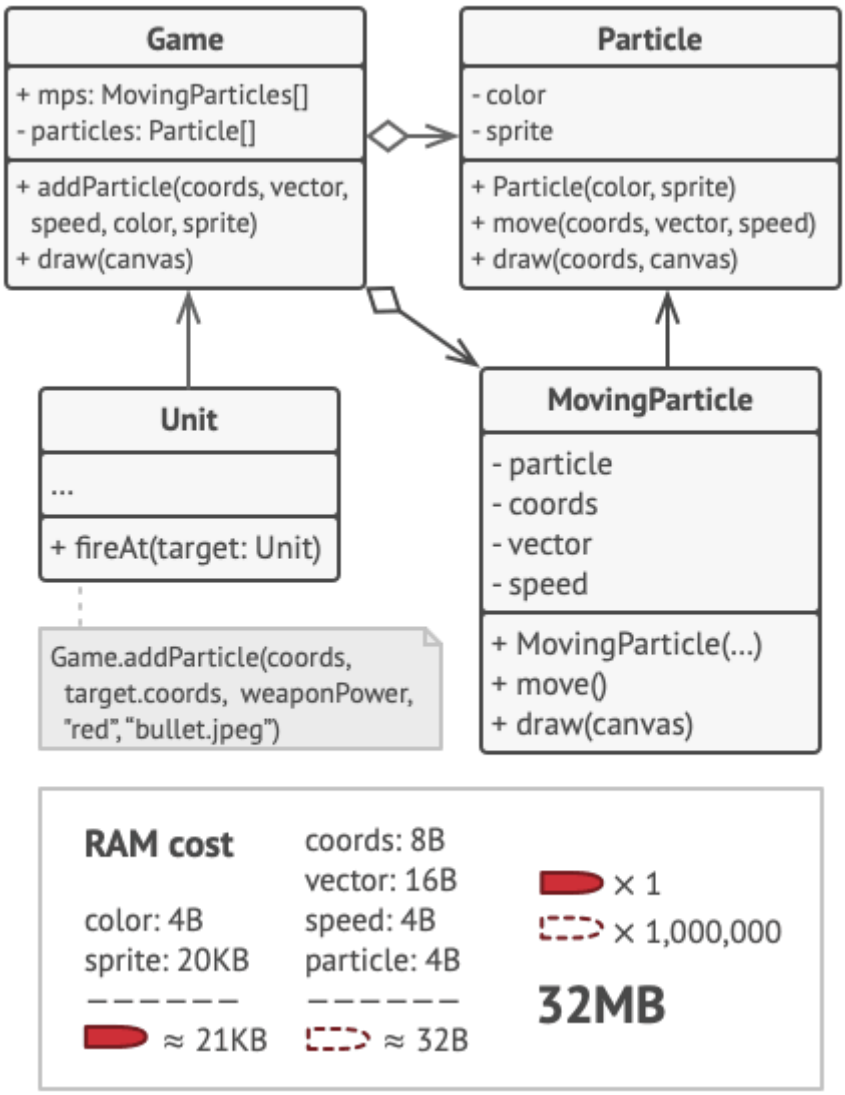


Andere Aspekte des Partikelzustands, wie Koordinaten, Bewegungsvektor und Geschwindigkeit, sind für jedes Partikel einzigartig. Schließlich ändern sich die Werte dieser Felder im Laufe der Zeit. Diese Daten repräsentieren den sich ständig ändernden

Kontext, in dem das Partikel existiert, während Farbe und Sprite für jedes Partikel konstant bleiben.

Diese konstanten Daten eines Objekts werden üblicherweise als *intrinsischer Zustand bezeichnet. Sie existieren innerhalb des Objekts; andere Objekte können sie nur lesen, nicht ändern. Der restliche Zustand des Objekts, der oft „von außen“ durch andere Objekte verändert wird, wird als extrinsischer Zustand* bezeichnet .

Das Flyweight-Muster schlägt vor, den extrinsischen Zustand nicht mehr im Objekt zu speichern. Stattdessen sollten Sie diesen Zustand an spezifische Methoden übergeben, die darauf angewiesen sind. Nur der intrinsische Zustand bleibt im Objekt, sodass Sie ihn in verschiedenen Kontexten wiederverwenden können. Dadurch benötigen Sie weniger dieser Objekte, da sie sich nur im intrinsischen Zustand unterscheiden, der deutlich weniger Variationen aufweist als der extrinsische.

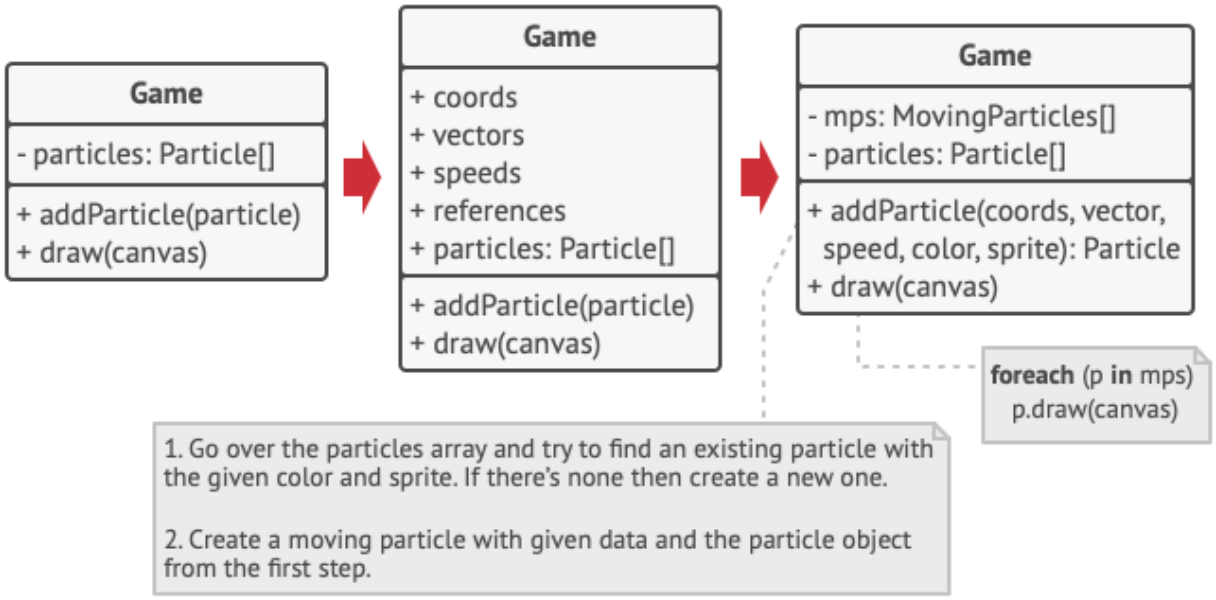


Kehren wir zu unserem Spiel zurück. Angenommen, wir hätten den extrinsischen Zustand aus unserer Partikelklasse extrahiert, würden nur drei verschiedene Objekte ausreichen, um alle Partikel im Spiel darzustellen: eine Kugel, eine Rakete und ein

Granatsplitter. Wie Sie wahrscheinlich schon erraten haben, heißt ein Objekt, das nur den intrinsischen Zustand speichert,ein Fliegengewicht.

Extrinsische Zustandsspeicherung
Wohin wird der extrinsische Zustand verschoben? Irgendeine Klasse sollte ihn doch speichern, oder? In den meisten Fällen wird er in das Containerobjekt verschoben, das Objekte aggregiert, bevor wir das Muster anwenden.

In unserem Fall ist dies das Hauptobjekt `Game`, das alle Partikel im `particles`Feld speichert. Um den extrinsischen Zustand in diese Klasse zu übertragen, müssen Sie mehrere Array-Felder erstellen, um Koordinaten, Vektoren und Geschwindigkeit jedes einzelnen Partikels zu speichern. Aber das ist noch nicht alles. Sie benötigen ein weiteres Array für die Speicherung von Referenzen auf ein bestimmtes Flyweight, das ein Partikel repräsentiert. Diese Arrays müssen synchron sein, damit Sie mit demselben Index auf alle Daten eines Partikels zugreifen können.



Eine elegantere Lösung besteht darin, eine separate Kontextklasse zu erstellen, die den extrinsischen Zustand zusammen mit dem Verweis auf das Flyweight-Objekt speichert. Dieser Ansatz würde nur ein einziges Array in der Containerklasse erfordern.

Moment mal! Brauchen wir nicht genauso viele dieser Kontextobjekte wie am Anfang? Technisch gesehen ja. Allerdings sind diese Objekte viel kleiner als zuvor. Die speicherintensivsten Felder wurden auf wenige Flyweight-Objekte verschoben. Jetzt können tausend kleine Kontextobjekte ein einziges schweres Flyweight-Objekt wiederverwenden, anstatt tausend Kopien seiner Daten zu speichern.

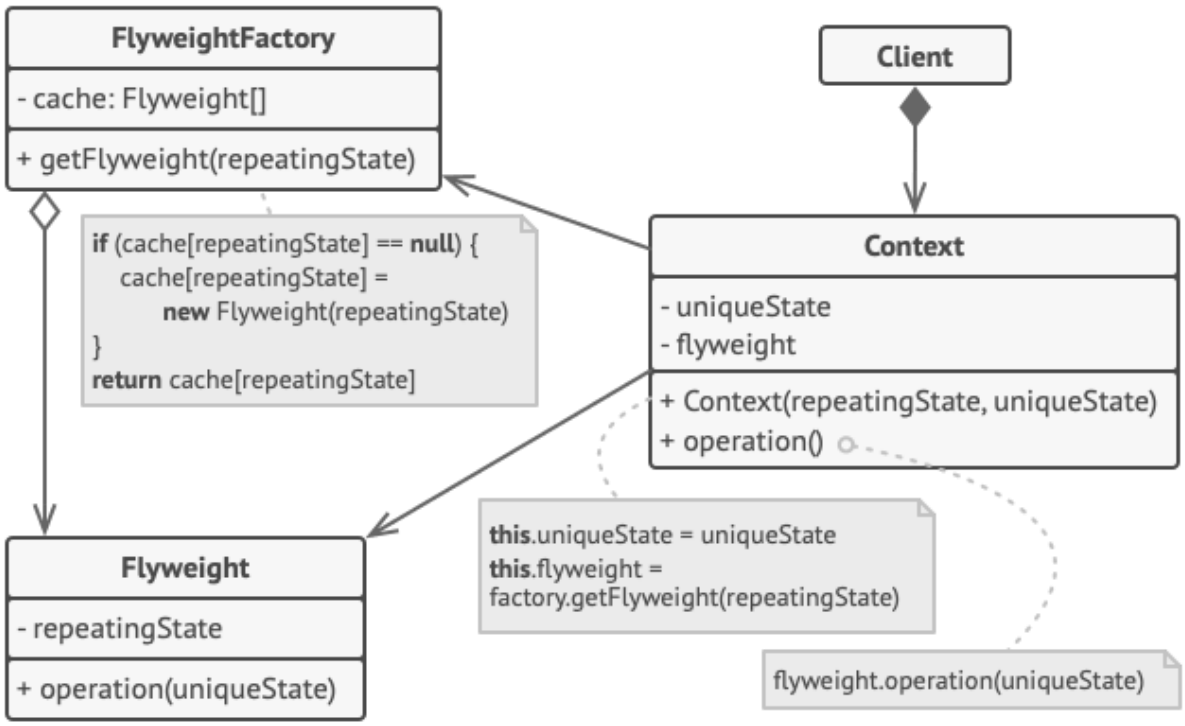
Fliegengewicht und Unveränderlichkeit
Da dasselbe Flyweight-Objekt in verschiedenen Kontexten verwendet werden kann, muss sichergestellt werden, dass sein Zustand nicht geändert werden kann. Ein

Flyweight sollte seinen Zustand nur einmal über Konstruktorparameter initialisieren. Es sollte keine Setter oder öffentlichen Felder für andere Objekte verfügbar machen.

Fliegengewichtsfabrik
Für einen bequemeren Zugriff auf verschiedene Flyweights können Sie eine Factory-Methode erstellen, die einen Pool vorhandener Flyweight-Objekte verwaltet. Die Methode akzeptiert den intrinsischen Zustand des gewünschten Flyweights von einem Client, sucht nach einem vorhandenen Flyweight-Objekt, das diesem Zustand entspricht, und gibt es zurück, falls es gefunden wurde. Andernfalls erstellt sie ein neues Flyweight und fügt es dem Pool hinzu.

Es gibt mehrere Möglichkeiten, diese Methode zu platzieren. Der naheliegendste Ort ist ein Flyweight-Container. Alternativ können Sie eine neue Factory-Klasse erstellen. Oder Sie können die Factory-Methode statisch machen und sie in eine echte Flyweight-Klasse einfügen.

Struktur

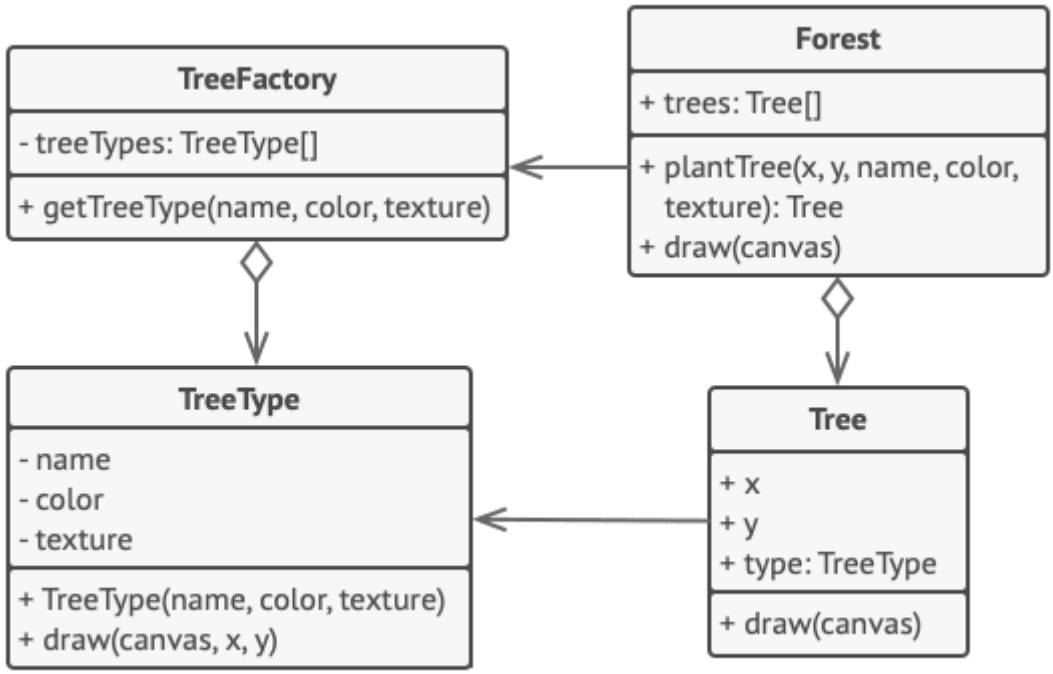


1. Das Flyweight-Muster ist lediglich eine Optimierung. Stellen Sie vor der Anwendung sicher, dass Ihr Programm tatsächlich das RAM-Verbrauchsproblem hat, das durch die gleichzeitige Speicherung einer großen Anzahl ähnlicher Objekte im Speicher entsteht. Stellen Sie sicher, dass dieses Problem nicht auf andere sinnvolle Weise gelöst werden kann.
2. Die **Flyweight** -Klasse enthält den Teil des Zustands des Originalobjekts, der von mehreren Objekten gemeinsam genutzt werden kann. Dasselbe Flyweight-Objekt kann in vielen verschiedenen Kontexten verwendet werden. Der in einem Flyweight gespeicherte Zustand wird als *intrinsisch bezeichnet*. Der an die Methoden des Flyweights übergebene Zustand wird als extrinsisch bezeichnet .

3. Die Klasse **Context** enthält den extrinsischen Zustand, der für alle Originalobjekte eindeutig ist. Wenn ein Kontext mit einem der Flyweight-Objekte gepaart ist, stellt er den vollständigen Zustand des Originalobjekts dar.
4. Normalerweise verbleibt das Verhalten des ursprünglichen Objekts in der Flyweight-Klasse. In diesem Fall muss der Aufrufer einer Flyweight-Methode auch entsprechende Teile des extrinsischen Zustands an die Methodenparameter übergeben. Alternativ kann das Verhalten in die Kontextklasse verschoben werden, die das verknüpfte Flyweight lediglich als Datenobjekt verwendet.
5. Der **Client** berechnet oder speichert den extrinsischen Zustand von Flyweights. Aus Sicht des Clients ist ein Flyweight ein Vorlagenobjekt, das zur Laufzeit konfiguriert werden kann, indem Kontextdaten in die Parameter seiner Methoden übergeben werden.
6. Die **Flyweight Factory** verwaltet einen Pool vorhandener Flyweights. Mit der Factory erstellen Clients keine Flyweights direkt. Stattdessen rufen sie die Factory auf und übergeben ihr Teile des intrinsischen Zustands des gewünschten Flyweights. Die Factory überprüft zuvor erstellte Flyweights und gibt entweder ein vorhandenes zurück, das den Suchkriterien entspricht, oder erstellt ein neues, wenn nichts gefunden wird.

Pseudocode

In diesem Beispiel hilft das **Flyweight** -Muster, den Speicherverbrauch beim Rendern von Millionen von Baumobjekten auf einer Leinwand zu reduzieren.



Das Muster extrahiert den sich wiederholenden intrinsischen Zustand aus einer Hauptklasse `Tree`und verschiebt ihn in die Flyweight-Klasse `TreeType`.

Anstatt dieselben Daten in mehreren Objekten zu speichern, werden sie nun in wenigen Flyweight-Objekten gespeichert und mit den entsprechenden `Tree`Objekten verknüpft, die als Kontexte fungieren. Der Client-Code erstellt mithilfe der Flyweight-Factory neue Baumobjekte. Dies reduziert die Komplexität der Suche nach dem richtigen Objekt und dessen Wiederverwendung bei Bedarf.

```
// The flyweight class contains a portion of the state of a
// tree. These fields store values that are unique for each
// particular tree. For instance, you won't find here the tree
// coordinates. But the texture and colors shared between many
// trees are here. Since this data is usually BIG, you'd waste a
// lot of memory by keeping it in each tree object. Instead, we
// can extract texture, color and other repeating data into a
// separate object which lots of individual tree objects can
// reference.
class TreeType is
    field name
    field color
    field texture
    constructor TreeType(name, color, texture) { ... }
    method draw(canvas, x, y) is
        // 1. Create a bitmap of a given type, color & texture.
        // 2. Draw the bitmap on the canvas at X and Y coords.

// Flyweight factory decides whether to re-use existing
// flyweight or to create a new object.
class TreeFactory is
    static field treeTypes: collection of tree types
    static method getTreeType(name, color, texture) is
        type = treeTypes.find(name, color, texture)
        if (type == null)
            type = new TreeType(name, color, texture)
            treeTypes.add(type)
        return type

// The contextual object contains the extrinsic part of the tree
// state. An application can create billions of these since they
// are pretty small: just two integer coordinates and one
// reference field.
class Tree is
    field x,y
    field type: TreeType
    constructor Tree(x, y, type) { ... }
    method draw(canvas) is
        type.draw(canvas, this.x, this.y)
```

```
// The Tree and the Forest classes are the flyweight's clients.
// You can merge them if you don't plan to develop the Tree
// class any further.
class Forest is
    field trees: collection of Trees

    method plantTree(x, y, name, color, texture) is
        type = TreeFactory.getTreeType(name, color, texture)
        tree = new Tree(x, y, type)
        trees.add(tree)

    method draw(canvas) is
        foreach (tree in trees) do
            tree.draw(canvas)
```

Anwendbarkeit

Verwenden Sie das Flyweight-Muster nur, wenn Ihr Programm eine große Anzahl von Objekten unterstützen muss, die kaum in den verfügbaren RAM passen.
Der Nutzen der Anwendung des Musters hängt stark davon ab, wie und wo es eingesetzt wird. Es ist besonders nützlich, wenn:

- Eine Anwendung muss eine große Anzahl ähnlicher Objekte erzeugen
- Dadurch wird der gesamte verfügbare RAM auf einem Zielgerät geleert
- Die Objekte enthalten doppelte Zustände, die extrahiert und zwischen mehreren Objekten geteilt werden können

So implementieren Sie

1. Teilen Sie die Felder einer Klasse, die zum Fliegengewicht wird, in zwei Teile auf:
    - der intrinsische Zustand: die Felder, die unveränderliche Daten enthalten, die über viele Objekte hinweg dupliziert sind
    - der extrinsische Zustand: die Felder, die für jedes Objekt eindeutige Kontextdaten enthalten
2. Belassen Sie die Felder, die den intrinsischen Zustand darstellen, in der Klasse, stellen Sie jedoch sicher, dass sie unveränderlich sind. Sie sollten ihre Anfangswerte nur innerhalb des Konstruktors annehmen.
3. Gehen Sie Methoden durch, die Felder des extrinsischen Zustands verwenden. Führen Sie für jedes in der Methode verwendete Feld einen neuen Parameter ein und verwenden Sie ihn anstelle des Felds.
4. Erstellen Sie optional eine Factory-Klasse zur Verwaltung des Flyweight-Pools. Diese sollte vor der Erstellung eines neuen Flyweights prüfen, ob ein vorhandenes Flyweight vorhanden ist. Sobald die Factory eingerichtet ist, können Clients Flyweights nur noch über sie anfordern. Sie sollten das gewünschte Flyweight beschreiben, indem sie seinen intrinsischen Zustand an die Factory übergeben.

5. Der Client muss Werte des extrinsischen Zustands (Kontext) speichern oder berechnen, um Methoden von Flyweight-Objekten aufrufen zu können. Der Einfachheit halber kann der extrinsische Zustand zusammen mit dem Flyweight-referenzierenden Feld in eine separate Kontextklasse verschoben werden.

Für und Wider

- Sie können viel RAM sparen, vorausgesetzt, Ihr Programm verfügt über zahlreiche ähnliche Objekte.

- Möglicherweise tauschen Sie RAM gegen CPU-Zyklen, wenn einige der Kontextdaten jedes Mal neu berechnet werden müssen, wenn jemand eine Flyweight-Methode aufruft.
- Der Code wird deutlich komplizierter. Neue Teammitglieder werden sich immer wieder fragen, warum der Status einer Entität auf diese Weise getrennt wurde.

Beziehungen zu anderen Mustern

- Sie können gemeinsam genutzte Blattknoten des **zusammengesetzten** Baums als **Flyweights** implementieren , um RAM zu sparen.
- **Flyweight** zeigt, wie man viele kleine Objekte erstellt, während **Facade** zeigt, wie man ein einzelnes Objekt erstellt, das ein ganzes Subsystem darstellt.
- **Flyweight würde Singleton** ähneln, wenn es gelänge, alle gemeinsamen Zustände der Objekte auf ein einziges Flyweight-Objekt zu reduzieren. Es gibt jedoch zwei grundlegende Unterschiede zwischen diesen Mustern:
    1. Es sollte nur eine Singleton-Instanz geben, während eine *Flyweight*- Klasse mehrere Instanzen mit unterschiedlichen intrinsischen Zuständen haben kann.
    2. Das *Singleton* -Objekt kann veränderlich sein. Flyweight-Objekte sind unveränderlich.